baseten

ИССЛЕДОВАНИЕ 12 ОКТЯБРЯ 2025 ГОДА

Пишите мало, учитесь вечно: LoRA первого ранга для непрерывного обучения

Почему обновления LoRA первого ранга могут стать недостающим звеном между статической тонкой настройкой и по-настоящему непрерывным обучением на графическом процессоре.

 Чарльз О'Нил (Базеттен)
 Макс Киркби (Baseten)
 Гарри Партридж (Бастетен)
 Джонатон Лю (Бастетен)

TL;DR

Предостережение в начале этого поста: на самом деле мы не знаем, стоит ли вообще рассматривать LoRA как способ непрерывного обучения. Изначально мы рассматривали этот вопрос с инженерной точки зрения: как сделать так, чтобы модель «ощущала», что она постоянно обновляется, а не представляет собой статичный снимок, полученный в результате дискретных циклов тонкой настройки? Мы хотели, чтобы непрерывное обучение больше походило на подсказывание: если у вас есть определенный обучающий сигнал, которому вы доверяете, или вы хотите его скорректировать или добавить новые обучающие сигналы, вы сразу увидите, как эти изменения начинают влиять на поведение модели. Непрерывное (или почти непрерывное) объединение адаптеров LoRA ранга 1 с моделью на тех же графических процессорах, на которых выполняется логический вывод, показалось нам правильным решением.

Однако это не решает многих проблем, связанных с непрерывным обучением. Например, хотя LoRA-модели страдают от «катастрофического забывания» в меньшей степени, чем при полной дообучении, мы знаем, что это все равно проблема. Кроме того, такой подход не всегда позволяет использовать

модульный подход, если вы постоянно объединяете LoRA-модели с базовой моделью. Например, если вы сохраняете отдельные LoRA-модели (а не постоянно обновляете их и объединяете), то можете, например, заменять LoRA-модели на одной и той же базовой модели, чтобы повысить эффективность логического вывода для нескольких клиентов. Это одна из причин, по которой мы уделяем больше внимания таким вещам, как КАРТРИДЖИ и тонкая настройка разреженной памяти, о которых мы расскажем в следующих публикациях.

Я думаю, что правильно будет исходить из того, что приведенные ниже рекомендации, скорее всего, сработают, если распределение задач будет стабильным и достаточно узконаправленным, чтобы можно было «залатать все дыры», постепенно корректируя обновления. Кроме того, вероятно, важно, чтобы примеры (в рамках задания) были достаточно повторяющимися и похожими друг на друга, чтобы не возникло неожиданных ситуаций, когда кто-то вдруг спросит, кто был президентом США в 1972 году, а вы все это время писали клинические заметки. (В целом, я считаю, что катастрофическое забывание для специализированных моделей — не такая большая проблема, как принято считать. Не то чтобы оно не происходило, но вам вряд ли важно, помнит ли ваша модель, кто был президентом в разные годы, если это никак не связано с текущей задачей.)

Видение

В начале этого года нас увлекла одна идея: что, если бы модели были более «живыми», как стажер Дваркаша, и изменения в подсказках ощущались бы мгновенно? Не статичные артефакты, которые периодически переобучаются, а нечто, что учится непрерывно, момент за моментом, — примерно так, как, по вашему мнению, происходит обучение человека. Мы хотели, чтобы работа с моделью воспринималась не как смена «снимков», а как диалог с чем-то, что активно учитывает обратную связь. Вы подаете обучающий сигнал, которому доверяете, возможно, корректируете его, добавляете новые сигналы, и видите, что эти изменения сразу же начинают влиять на поведение модели — не после ночного обучения, а прямо в процессе работы с ней.

Это привело нас в тупик. Мы задумались: хорошо, допустим, мы хотим постоянно объединять адаптеры LoRA с рангом 1 в модель на тех же графических процессорах, которые выполняют логический вывод. Что это значит с математической точки зрения? Можно ли вообще это сделать, чтобы ничего не развалилось?

Первая Подсказка

Когда вы обучаете нейронную сеть методом стохастического градиентного спуска, вы уже выполняете обновления низкого ранга. Не приблизительно или эффективно, а буквально. Подумайте о том, что происходит, когда один обучающий пример проходит через линейный слой. У вас есть преобразование $h = Wx$, где $W \in \mathbb{R}^{m \times n}$, а $x \in \mathbb{R}^n$ — входные данные. Во время обратного распространения ошибки функция потерь ℓ посылает сигнал $\delta = \partial \ell / \partial h \in \mathbb{R}^m$. Градиент по отношению к матрице весов становится равным:

$$G = \nabla_W \ell = \delta x^\top$$

This is just an outer product—a rank-1 matrix. Every column of G is a scalar multiple of δ, every row is a scalar multiple of $x^\top$. Even with a mini-batch $(\delta_b, x_b)_{b=1}^B$, you're just summing these up:

$$G_{\text{batch}} = \sum_{b=1}^{B} \delta_b x_b^\top$$

I guess intuition says that restricting yourself to rank-1 updates should cripple learning (even though people have written about this being naturally low-rank before); after all, you're only pushing weights along a single direction. But if SGD is already doing this naturally, maybe we weren't restricting anything fundamental. Maybe we were just making explicit what the optimization process already wanted to do.

Low-rank updates

We kept wondering, if we're going to merge these rank-1 updates continuously, do we need to be super careful about when we merge? Like, what's the difference between accumulating a bunch of micro-updates and merging them all at once versus merging after each tiny step?

It turns out they're basically the same thing up to second-order terms. When each micro-loss $\ell_i(W)$ is L-smooth and your updates are small, you can expand the gradient via Taylor series. If $g_i(W) = \nabla \ell_i(W)$, then:

$$g_i(W_{i-1}) = g_i(W) + H_i(W)[W_{i-1} - W] + O(|W_{i-1} - W|^2)$$

Since the drift $W_{i-1} - W$ is itself $O(\eta)$, the correction per micro-update is $O(\eta)$. Multiplying by η to get the actual step gives you $O(\eta^2)$. Summing over all steps, you get $|W_{\text{merge}} - W_{\text{acc}}|_F = O(\eta^2)$.

What this means in practice is that when you move the base weights by tiny amounts, the gradient at the next micro-step barely changes to first order. Whether you merge immediately or bundle updates, you get essentially the same answer.

Following the Thread

Okay, but if you're only doing rank-1 updates, how do you ever learn anything complex? The answer lies in how these updates accumulate over time. Each micro-update can be written $\hat{\Delta}_t = \gamma_t \hat{u}_t \hat{v}_t^\top$ with unit vectors and magnitude $\gamma_t > 0$. After T merges:

$$\Delta_{\leq T} = \sum_{t=1}^{T} \gamma_t \hat{\Delta}_t$$

The key insight is that the span of your updates grows as gradients naturally rotate through different directions. We started thinking about this through the lens of stable rank:

$$\mathrm{srank}(\Delta) = |\Delta|_F^2 / |\Delta|_2^2$$

If updates keep pointing in fresh directions (bounded coherence $\max_{s<t} |\langle \hat{\Delta}_t, \hat{\Delta}_s \rangle_F| \leq \rho < 1$, then:

$$\mathrm{srank}(\Delta_{\leq T}) \gtrsim T/(1 + \rho(T-1))$$

When ρ is small, stable rank grows roughly linearly with T. It's like you're painting a picture one brushstroke at a time; each stroke is simple, but they accumulate into something rich. The restriction is momentary; the capacity is cumulative.

Deeper Implications

This led us to think about what LoRA is actually doing geometrically. When you parameterise an increment as $\Delta W = AB^\top$, you're essentially solving a proximal problem:

$$\min_{\mathrm{rank}(\Delta) \leq r} \langle \nabla \ell(W), \Delta \rangle + \frac{1}{2\eta} |\Delta|_F^2$$

The solution comes from the Eckart–Young–Mirsky theorem. If you decompose the gradient as $\nabla \ell(W) = \sum_j \sigma_j u_j v_j^\top$, the best rank-r approximation keeps the top r components:

$$\Delta^* = -\eta \sum_{j=1}^{r} \sigma_j u_j v_j^\top$$

For rank-1, you're literally just picking the single most important direction at each step. It's steepest descent in the geometry of the nuclear-norm ball rather than regular Euclidean space. There's something philosophically satisfying about this because you're not trying to do everything at once, just the most important thing right now.

Where It Gets Messy

Of course, not everything is this clean. The place where our beautiful continuous learning vision starts to get complicated is when you move from supervised fine-tuning to reinforcement learning.

In SFT with teacher forcing, your data $x \sim p_{\text{data}}$ doesn't depend on model weights. Update W mid-epoch, and your gradient still targets $\mathbb{E}x \sim p_{\text{data}}[\ell(W; x)]$. There's no notion of being "off-policy." This is why our first-order equivalence holds completely, because the data distribution stays fixed regardless of when you merge. This is where you really get that magical "feels like prompting" experience we were after.

But in on-policy RL, your data $tau \sim p(\tau|\pi_W)$depends directly on W. If you merge updates during rollout collection, you're changing the policy generating later trajectories. Now you need importance weights:

$$\prod_t \left(\pi_W(a_t|s_t)/\pi_{W_t}(a_t|s_t)\right)$$

There's also this striking difference in information density that we kept coming back to. In SFT, a sequence of length T with cross-entropy b bits per token gives you roughly bT bits of training signal. In terminal-reward REINFORCE, an entire episode might communicate less than one bit about the first decision. This actually explains something we'd noticed empirically ie rank-1 often matches full fine-tuning in RL not because it's equally expressive, but because the supervision channel itself is the bottleneck (also heaps of other people have talked about this).

But really the hardest thing is managing the optimizer. Using Adam for continuously merging LoRAs in this fashion is non-trivial because Adam's updates are guided by the first and second moments accumulated over previous steps. After you merge a LoRA adapter and initialize a fresh one, if you keep using the old gradient moments, they'll push the new adapter in the same direction as the previous one ie you end up optimizing the same subspace repeatedly. ReLoRA tackles this with a partial reset of the optimizer state

through magnitude pruning, zeroing out the learning rate with a warm-up to prevent the loss from exploding, and even requiring a short full-rank warm-start when training from scratch. This is probably the biggest engineering headache in making this stuff actually work.

What This Looks Like on the GPUs

We kept coming back to this engineering reality: what does it actually mean to run continuous learning on the same GPUs serving production traffic? This is where rank-1 LoRA starts to look less like a nice mathematical property and more like the only thing that could possibly work.

Think about the memory constraints first. Full fine-tuning needs optimizer state for every parameter—with Adam, that's roughly 3× the model size. For a 70B model, you're looking at 210B parameters worth of optimizer state alone. Rank-1 LoRA per layer? You're storing just $A \in \mathbb{R}^{m\times 1}$ and $B \in \mathbb{R}^{n\times 1}$ plus their tiny optimizer states. Memory scales as $O(m+n)$ instead of $O(mn)$.

But here's where we had to get precise about what "continuous updating" actually means. There are two totally different operations that we kept conflating at first:

Hot-swapping adapters is the instant gratification part. You keep the adapters factored and compute $Wx + B(Ax)$ at inference, and the cost is just $O(r(d_{in} + d_{out}))$ extra ops per token, which with $r = 1$ is basically nothing. These A, B tensors are so small you can atomically swap pointers between requests with microsecond-scale locks. Broadcast the new tensors across your tensor-parallel shards once, without gigabyte copies and repacking. This is how you get that immediate “the model just learned something” feeling without touching the base weights at all.

1. **Actually merging is different.** Adding AB^T into W means touching $O(mn)$ elements per matrix—you're doing this for Q/K/V/O and MLP layers across all shards. It's bandwidth-bound, not compute-bound, and definitely not free. If you're running quantised (which, let's be honest, you probably are), you either dequantise-add-requantise or maintain a high-precision delta buffer that gets folded in during periodic requantisation.

So the production pattern we started imagining looks like this: serve with factored adapters for immediate effect. Accumulate gradients on those adapters right on the serving GPUs. When you've got a moment (low load, or on a shadow replica) run the actual merge using fused rank-1 kernels. Roll traffic to the merged version. Keep a ring buffer of recent deltas for rollback.

Picture it: 8 H100s serving your model, each shard keeping a few kilobytes of adapter parameters pinned in HBM. User interactions immediately update these tiny adapters via

NCCL broadcast. Every few seconds, or when load dips, you pick a shadow replica, apply the rank-1 update to its weights (respecting quantisation), smoke test, and flip traffic. From the outside, the model just seems to be getting smarter continuously. You can change your learning signals (swap reward functions, adjust your LLM-as-judge) and feel the model start responding differently almost immediately through the adapter path, which is just so cool to think about.

There's something elegant about checkpointing here too. Ring-buffer the last K rank-1 updates with metadata about what produced them. Something goes wrong? Roll back merges. Want to understand why the model changed? The provenance is right there, not in some terabyte checkpoint graveyard.

The reality check is that companies like Cursor already redeploy every two hours, which is incredible engineering. But what we're sketching is qualitatively different ie behaviour changes immediately via adapters, persists via background merges, and the serving fleet just keeps humming. It's learning that feels continuous because it actually is continuous, with updates flowing as naturally as forward passes.

Technical Musings

There's this whole question of capacity that kept nagging at us. People talk about parameter counts, but that feels like the wrong lens. If you train on N tokens at b bits per token, you can't extract more than bN bits of generalisable information. A rank-1 LoRA across all projections of a modern model is millions of parameters. Even at 1–2 bits of information per parameter, that's massive headroom for most continual learning scenarios.

We also played around with LoRA-XS, which takes a different philosophical stance. You approximate W via truncated SVD as $W \approx U_r \Sigma_r V_r^\top$ and learn just a small core matrix $R \in \mathbb{R}^{r \times r}$. The updates become $\Delta W = U_r R V_r^\top$. It's incredibly parameter-efficient, just r^2 parameters—and great for composition. But you lose that span growth we talked about. Your reachable span is frozen to what the pre-trained weights already encode. Whether that's a feature or a bug depends on your use case. But we've found it doesn't work that well.

As for convergence, in convex settings you can view rank-1 LoRA through Frank-Wolfe optimization on the nuclear-norm ball—you get $O(1/t)$ convergence. For the non-convex networks we actually care about, with appropriate step sizes you still get:

$$\min_{t<T} |\nabla \ell(W_t)| = O(1/\sqrt{T})$$

Nobody's claiming you'll land at exactly the same minimizer as full fine-tuning, but you reach a comparable stationary point.

Dwarkesh's comparison of LoRA vs ICL

Dwarkesh raised an interesting point recently, comparing bytes in KV cache versus LoRA adapters, noting that at 100k tokens you get 37× "compression" and wondering if this meant LoRA couldn't match in-context learning's richness. But this comparison tracks the wrong thing entirely. KV cache is read memory: transient, lossless storage that grows linearly with tokens so attention can query any context. LoRA is write memory: a fixed, tiny set of weights storing generalizations that gradient descent has distilled. Comparing their bytes per token is like comparing the size of your desk to the size of your brain, because they serve fundamentally different purposes.

The real question isn't "how many bytes per token" but "how many bits of generalizable signal does a session provide?" In supervised fine-tuning, a sequence of length T at b bits/token carries up to bT bits of learning signal. In binary RL, an entire episode might communicate one bit (it's a bit handwavy, some of these information theoretic arguments, but we move). Most sessions contain far fewer bits-that-matter than the raw token count suggests: a rule to forbid, a corrected schema, a preferred template, compact invariants. A rank-1 LoRA across all projections has millions of parameters, massive headroom for these bits. The bottleneck is almost never adapter capacity; it's the rate at which you extract useful signal from your data stream. And importantly, unlike KV cache which vanishes between sessions, those learned bits persist forever, which is the one thing in-context learning cannot give you.

For attribution in academic contexts, please cite this work as:

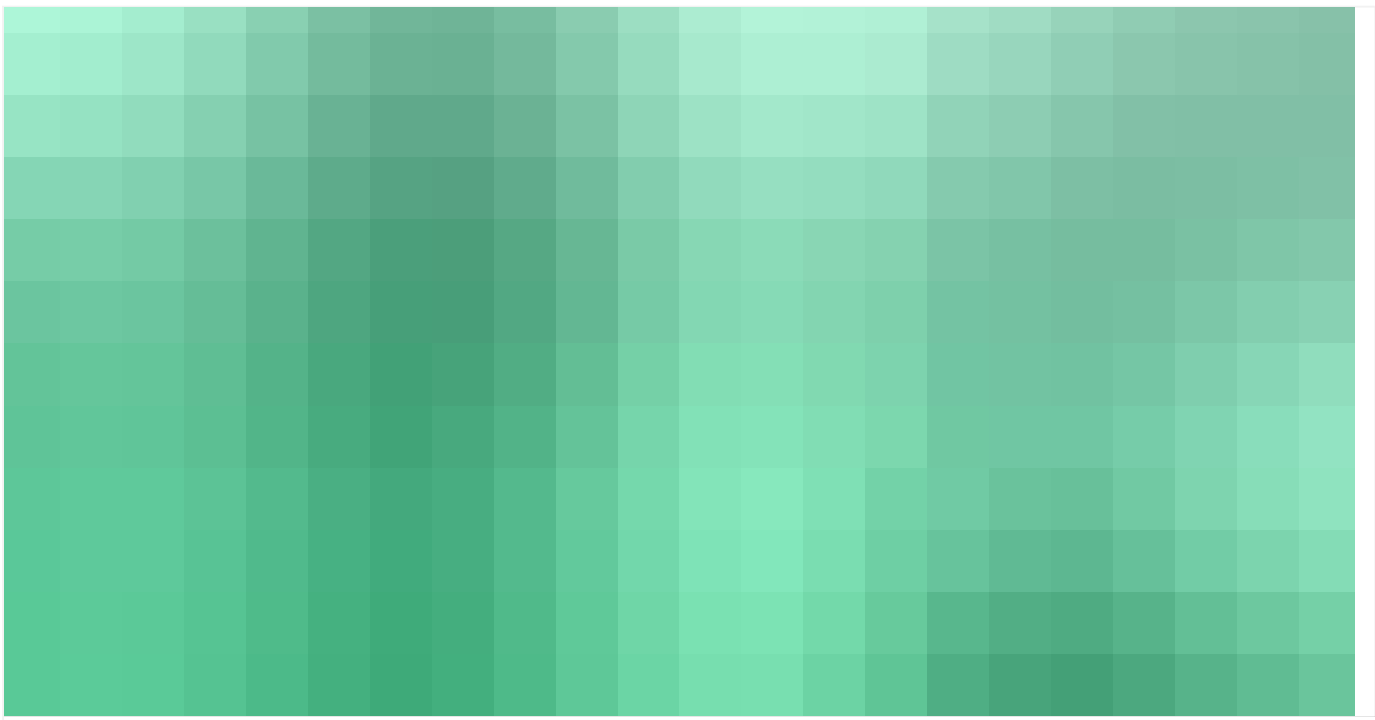
O'Neill, Charles, Max Kirkby, Harry Partridge, and Jonathon Liu. "Write Small, Learn Forever: Rank-1 LoRA for Continual Learning." Baseten Research, October 12, 2025. https://www.baseten.co/research/write-small-learn-forever/

BibTeX citation

```
@misc{oneill2025lora,
  author       = {O'Neill, Charles and Kirkby, Max and Partridge, Harry and
Liu, Jonathon},
  title        = {Write Small, Learn Forever: Rank-1 {LoRA} for Continual
Learning},
  year         = {2025},
  month        = oct,
  day          = {12},
  howpublished = {Baseten Research},
```

```
  url          = {https://www.baseten.co/research/write-small-learn-forever/},
}
```

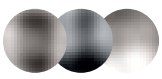
Other research



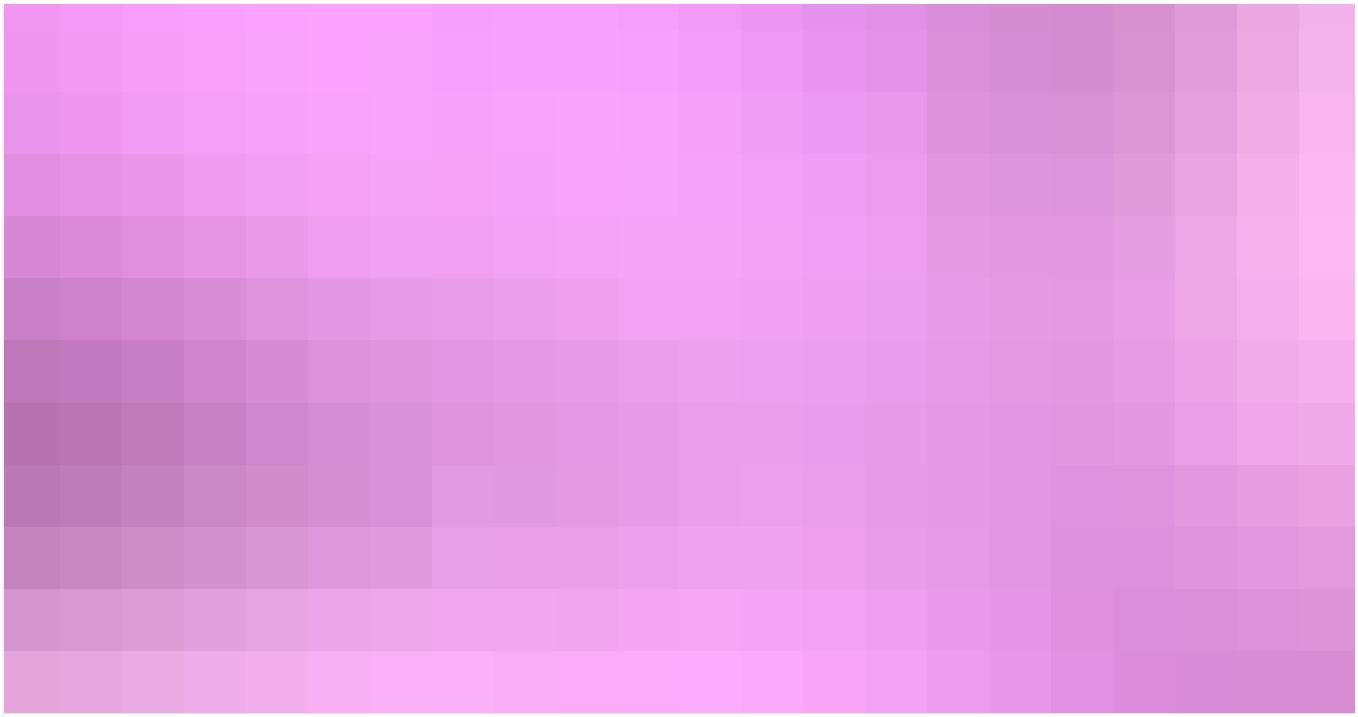
RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



RESEARCH

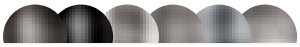
Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.

RESEARCH

Post-training frontier legal agents with Baseten Research

Post-training a 27B open-weight model to the frontier on Harvey LAB with Baseten Research



CHARLES O'NEILL • 5 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference

Model APIs

Training

Frontier Gateway

Baseten Inference Platform

Model Runtimes

Infrastructure

Multi-cloud

Deployment options

Cloud

Self-hosted

Hybrid

Embedded engineering

Forward deployed engineers

Modalities

Transcription
Image Generation
Text-to-speech
Large language models
Compound AI
Embeddings

Industries

Enterprise
Healthcare

Developer

Model library
Documentation
Changelog

Resources

Research
Customers
Blog
Guides
Events
Partners
Savings calculator
Trust Center
About us
Startups
Careers
Contact us

Popular models

GLM 5.2
Kimi K2.7 Code
DeepSeek V4
GPT OSS 120B
Whisper Large V3
NVIDIA Nemotron 3 Ultra
Explore all

Legal

Terms and Conditions
Privacy Policy
Service Level Agreement

ALL SYSTEMS NORMAL